Table 2. Nodes in largest backtrack set / wakeup tree, memory and time usage of the source- and optimal-DPOR algorithms (as implemented in Concuerror) for the program in Fig. 12.

		source			optimal		
n	Traces Explored	Nodes	Memory	Time	Nodes	Memory	Time
1	2	2	22MB	0.01s	7	22MB	0.01s
2	4	3	22MB	0.02s	21	22MB	0.02s
3	12	4	22MB	0.05s	79	22MB	0.06s
4	48	5	22MB	0.24s	374	23MB	0.24s
5	240	6	22MB	1.21s	2159	27MB	1.23s
6	1440	7	22MB	8.74s	14698	42MB	8.96s
7	10080	8	22MB	1m02s	115091	114MB	1m07s
8	80640	9	22MB	9m30s	1018094	801MB	9m48s
9	725760	10	22MB	89m14s	9916765	8364MB	99m2s

10 IMPLEMENTATION

In this section, we describe our implementation in the context of Concuerror, a stateless model checking tool for Erlang. Erlang is an industrially relevant programming language based on the actor model of concurrency [Armstrong 2010]. In Erlang, actors are realized by language-level processes implemented by the runtime system instead of being directly mapped to operating system threads. Each Erlang process has its own private memory area (stack, heap and mailbox) and communicates with other processes via message passing. A call to the `spawn` function creates a new process p and returns a *process identifier* (PID) that can be used to send messages to p. Messages are sent *asynchronously* using the `!` operation (or `send` function). Messages get placed in the mailbox of the receiving process in the order they arrive. A process can then consume messages using *selective* pattern matching in `receive` expressions, which are *blocking* operations when a process mailbox does not contain any matching message. Optionally, a `receive` may contain an `after` clause which specifies a *timeout* value (either an integer or the special value `infinity`) and a value to be returned if the timeout time (in ms) is exhausted.

Erlang processes do not share any memory by default. Still, the Erlang implementation comes with a key-value store mechanism, called *Erlang Term Storage (ETS)*, that allows processes to create memory areas where terms shared between processes can be inserted, looked up, and updated. Such areas are the ETS tables that are explicitly declared `public`. The runtime system automatically serializes accesses to these tables when this is necessary and also comes with mechanisms that guarantee atomicity of some operations (e.g., a bulk `insert`). Each ETS table is owned by the process that created it and its memory is reclaimed by the runtime system when this process exits if no other process has inherited this table.

Erlang has all the ingredients needed for concurrency via message passing and most of the ingredients (e.g., reads and writes to shared data, etc.) needed for concurrent programming using shared memory. Unsurprisingly, Erlang programs are prone to "the usual" errors associated with concurrent execution, although the majority of them revolves around message passing and misuse of built-in primitives implemented in C.

Figure 13 shows the program of Fig. 1 written in Erlang, generalized to `N` instead of just two readers. On line 5, a public ETS table named `tab` is created and is shared between `N+1` processes: `N` readers and one writer. The writer inserts a key-value pair, using `k` as a key. Each of the `N` readers